



INTRODUCCIÓN A LOS SISTEMAS DISTRIBUIDOS  
(75.43/75.33/95.60)

TRABAJO PRÁCTICO N° 1  
**File Transfer Protocol**

*Grupo N°11*

Profesor:  
Dr. Ing. Jose Ignacio Alvarez Hameli

5 de mayo de 2026

# Índice

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>1. Introducción</b>                               | <b>2</b>  |
| <b>2. Hipótesis y suposiciones realizadas</b>        | <b>2</b>  |
| <b>3. Implementación</b>                             | <b>3</b>  |
| 3.1. Uso de sockets . . . . .                        | 3         |
| 3.2. Diseño de FTP-Header . . . . .                  | 4         |
| 3.3. Mecanismos de error-recovery . . . . .          | 5         |
| 3.3.1. Stop&Wait . . . . .                           | 6         |
| 3.3.2. Selective Repeat . . . . .                    | 7         |
| 3.4. Manejo de errores . . . . .                     | 9         |
| 3.5. Concurrencia . . . . .                          | 10        |
| <b>4. Pruebas</b>                                    | <b>11</b> |
| 4.1. Stop&Wait . . . . .                             | 12        |
| 4.2. Selective Repeat . . . . .                      | 17        |
| 4.3. Resultados . . . . .                            | 19        |
| <b>5. Preguntas a responder</b>                      | <b>20</b> |
| 5.1. Modelo Cliente-Servidor . . . . .               | 20        |
| 5.2. Función de un Protocolo de Aplicación . . . . . | 21        |
| 5.3. Otras características de FTP . . . . .          | 22        |
| 5.3.1. Sintaxis de los mensajes . . . . .            | 22        |
| 5.3.2. Estrategia de fragmentación . . . . .         | 23        |
| 5.3.3. Maquina de estados y ciclo de vida . . . . .  | 23        |
| 5.4. Protocolo UDP vs. TCP . . . . .                 | 23        |
| 5.5. Stop&Wait vs. Selective Repeat . . . . .        | 24        |
| <b>6. Dificultades encontradas</b>                   | <b>24</b> |
| 6.1. Posibles mejoras . . . . .                      | 25        |
| <b>7. Conclusión</b>                                 | <b>26</b> |
| <b>8. Referencias</b>                                | <b>27</b> |

## 1. Introducción

El Internet como herramienta es cada día más común y alcanza a sus usuarios nuevas alternativas de uso de forma diaria. Este fenómeno genera que el estudio del funcionamiento de las redes informáticas sea cada vez más substancial para el avance en diversas disciplinas que incluso pudieron tener poco en común con la computación en el pasado.

En la formación de un ingeniero informático es fundamental tener conocimiento sobre los distintos componentes que conforman el Internet, permitiendo a partir de este, resolver problemas y presentar alternativas innovadoras que ayuden a la red crecer y mejorarse, ya sea en cuestiones de seguridad, velocidad o funcionalidad.

En el presente trabajo se podrán en práctica los conocimientos adquiridos hasta el momento respecto al funcionamiento de la red y el modelo TCP/IP, realizando una implementación de un protocolo de capa de aplicación llamado `file transfer protocol` (que llamaremos *FTP*) a partir del encapsulamiento del `user datagram protocol`, también llamado *UDP*, haciendo que este admita la conexión y añadiendo funcionalidad para hacer uso del mecanismo de error-recovery de *Selective Repeat*.

Se presentarán hipótesis sobre el funcionamiento de dicho protocolo, comparando el mecanismo de error-recovery utilizado con la alternativa de *Stop&Wait* también implementada y como una opción debe de ser más óptima en el uso de recursos temporales que otra. Se hará un análisis en profundidad del protocolo usando la herramienta de *mininet* para poder simular una conexión por fuera del `localhost`. Por último se obtendrá una conclusión a partir de las hipótesis presentadas y los resultados de la implementación de *FTP*.

## 2. Hipótesis y suposiciones realizadas

Se espera que dentro de ambas configuraciones de error-recovery, el *Stop&Wait* tenga una diferencia substancial de tiempo de ejecución mayor que en el uso del *Selective Repeat* dado al hecho que en este último se envían una mayor cantidad de paquetes dentro de un mismo periodo.

Además usando el protocolo de recuperación de errores de repetición selectiva se busca, en un periodo de dos minutos, mandar sin problemas un archivo de 15MB de tamaño en una red inestable con una perdida del 10 %

de los paquetes enviados. En promedio, un 10 % de pérdida de paquetes significa que cada paquete se enviará 1,11 veces lo que significa que para un archivo binario de 15MB que equivalen a 15,728,640 bytes particionado en 11,235 segmentos de 1400 bytes<sup>1</sup> se realizarían 12.471 transmisiones.

Dada la restricción de 2 minutos para completar la transferencia, la implementación debe ser capaz de sostener un ritmo promedio de 104 paquetes por segundo. En consecuencia, la hipótesis de este trabajo sostiene que el tamaño de la ventana de envío es el parámetro crítico: una ventana optimizada permitirá 'llenar el canal' y seguir transmitiendo datos nuevos mientras se aguarda el timeout de los segmentos perdidos, logrando así que el tiempo total de transmisión sea inferior al límite de 120 segundos.

Adicionalmente, se supone que el cliente que busca descargar un archivo desde el servidor conoce la cantidad de espacio de almacenamiento disponible presente en su dispositivo en el momento de la descarga. De no tener espacio disponible se levantará una Excepción del tipo `OSError` (propia del método `write()` de Python) y se capturará dicha Excepción, cerrando la conexión.

### 3. Implementación

El *protocolo FTP* se basa en la arquitectura Cliente-Servidor, utilizando el protocolo UDP como protocolo de transporte. Este protocolo se implementó usando el lenguaje de programación Python en su versión 3.13.5 utilizando Github para permitir el trabajo colaborativo así como también la revisión de código anterior a la integración dentro de la rama principal de trabajo. Además, no se utilizaron librerías externas a las librerías nativas de Python para la realización de este protocolo.

#### 3.1. Uso de sockets

La infraestructura de socket se refiere a una estructura de software, proporcionada por el sistema operativo, que permite la comunicación bidireccional entre procesos, ya sea en la misma máquina o a través de la red. Actúa como un *endpoint* (del inglés, punto final) en una conexión, facilitando el intercambio de datos entre Cliente y Servidor. Un socket es una combinación

---

<sup>1</sup>La razón de multiplexación en segmentos de 1400B se explorará en profundidad en la sección sobre el desarrollo del trabajo

entre una dirección IP, un protocolo de transporte (en este caso UDP) y un número de puerto, el Servidor espera una conexión y uno o más clientes se conectan a él creandose así un socket por cliente para comenzar el intercambio de información.

En este protocolo se usó la librería nativa de Python `socket` que proporciona un conjunto de funciones que permite a la aplicación crear y gestionar la comunicación entre procesos. `socket` contiene una constante llamada `SOCK_DGRAM` que al crear un socket utiliza el protocolo de transporte UDP. UDP es conocido por ser sin conexión, lo que significa que el paquete que se envía por medio de UDP no se reporta al emisor si llegó correctamente o no. La decisión de usar este protocolo de transporte fue hecha dado a que presenta una limitación importante a la hora de tener que implementar el *protocolo FTP*, ya que normalmente para una buena transferencia de archivos debería utilizarse un protocolo más robusto como TCP, por lo que el protocolo implementado debe compensar ante la falta de acuse de recibo presente en UDP.

## 3.2. Diseño de FTP-Header

La estructura de header de este protocolo se basó en la posibilidad de poder realizar la conexión entre dos procesos similar a como se realiza dentro del *Transmission Control Protocol* (TCP), además de permitir el manejo de errores que pueda ocurrir dentro la ventana envío de paquetes en la transmisión del archivo de un proceso a otro. Es por esta razón que se adicionó al header del protocolo dos parámetros, un número de secuencia y reconocimiento relativos al grupo de paquetes enviados para que al enviarse un paquete, el receptor del paquete pueda avisar al emisor que se recibió la información con normalidad. Estos parámetros, resumidos en las siglas **SEQ** y **ACK**<sup>2</sup>, son complementarios ya que al enviarse un paquete se envía con un cierto número **SEQ** y es trabajo del receptor notificar con el **ACK** del mismo número que recibió el paquete.

Otra decisión de diseño que se tomó a la hora de crear el FTP-Header fue la de incorporar un *2-way handshake* al comienzo de la conexión. En este, el Cliente envía un paquete especial al Servidor para verificar que el destinatario se encuentre disponible para la recepción. Además en este proceso de verificación se aprovecha el envío de este paquete especial para enviar otras

---

<sup>2</sup>Del inglés *sequence* y *acknowledgement*, secuencia y reconocimiento respectivamente

características de la conexión dentro del *payload* del paquete así minimizar la cantidad de paquetes enviados previos al envío de datos del archivo. Una vez que el Servidor confirma la recepción sin errores de este primer *Paquete de Handshake* se comienza la transmisión del archivo.

Para admitir este *2-way handshake* se agregó un grupo de flags que representan distintos tipos de paquete que pueden enviarse a lo largo de la transmisión:

- **Handshake:** Usado durante el *2-way Handshake* para que el Cliente sepa que el servidor está disponible y para que el Servidor sepa que hay otro Cliente queriendo transmitirle información.
- **Data:** Esta flag avisa a quien esté recibiendo el archivo que este paquete contiene datos del archivo a recibir.
- **Acknowledge N:** Un paquete de este tipo avisa al emisor de los datos de archivo que el segmento de archivo particionado que envió con número de secuencia  $N$  llegó con normalidad y no debe retransmitirse.
- **Finalize:** El ultimo paquete que envía el emisor del archivo, cierra la conexión con el receptor y avisa a este que no hay más archivos por enviarse.

Se optó por un *2-way handshake* debido a que la relación emisor-receptor está predefinida y es estática. Si bien esto introduce vulnerabilidades ante paquetes **Handshake** duplicados o retrasados, se considera aceptable para el alcance de este trabajo, delegando la robustez a los reintentos de la capa de aplicación y asumiendo que el receptor descartará paquetes fuera de la ventana inicial.

Por ultimo, se agregó la longitud del header dentro de este para cuando se debe buscar el payload del paquete se pueda encontrar con exactitud el lugar donde comienzan los datos y un checksum para el control de corrupción de datos, que se discutirá más en profundidad en una sección posterior.

### 3.3. Mecanismos de error-recovery

Dado que a lo largo de la transmisión de datos pueden perderse paquetes *en vuelo*<sup>3</sup>, se debe implementar mecanismos que en casos donde se pierda

---

<sup>3</sup>Incluso se simulará la perdida de paquetes para poder probar la implementación de los mecanismos descritos en esta sección

algún paquete de datos se pueda notificar al emisor del archivo que debe reenviar dicho segmento perdido.

### 3.3.1. Stop&Wait

En una primera etapa se implementó el mecanismo de recuperación de errores Stop&Wait. La razón de implementar un mecanismo como Stop&Wait es para poder comparar distintos protocolos de error-recovery y determinar cual de ellos es el mejor respecto al tiempo de transferencia. En el caso de este trabajo se usará el Stop&Wait como marco de referencia debido a que es el mecanismo más simple de control de errores.

**Stop&Wait** El emisor envía un paquete y espera al acuse de recibo (ACK) de su receptor para enviar el siguiente. En caso de no recibirse el ACK del lado del receptor de la información, se retransmite el último paquete hasta recibirse el reconocimiento de recibo correcto, así garantizando la integridad de los datos enviados.

Como se muestra en la Figura 1, el funcionamiento normal del Stop&Wait garantiza que el emisor sepa que el paquete que acaba de enviar se recibió correctamente.

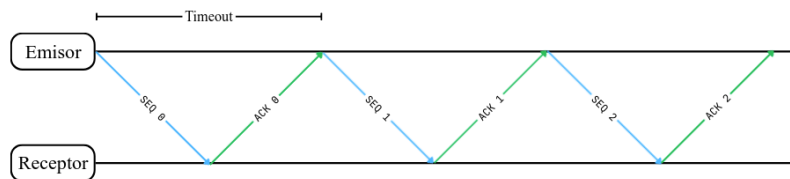


Figura 1: Función normal del Stop&Wait.

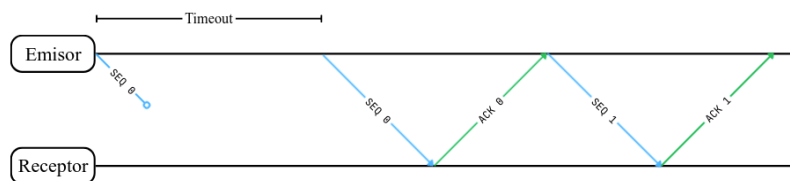


Figura 2: Retransmisión por pérdida de paquete de datos.

Para los casos donde el emisor no puede garantizar que el paquete se recibió con normalidad, se realiza una retransmisión de dicho paquete, incluso si en un futuro el acuse de recibo llega al emisor.

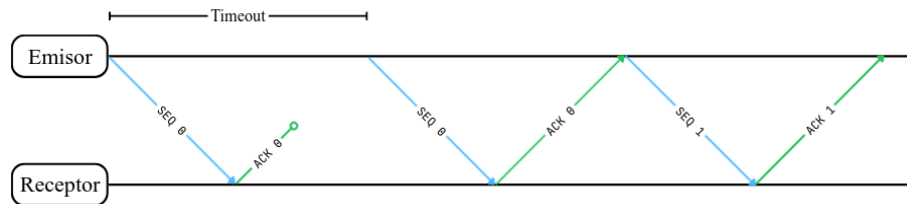


Figura 3: Retransmisión por pérdida de acuse de recibo.

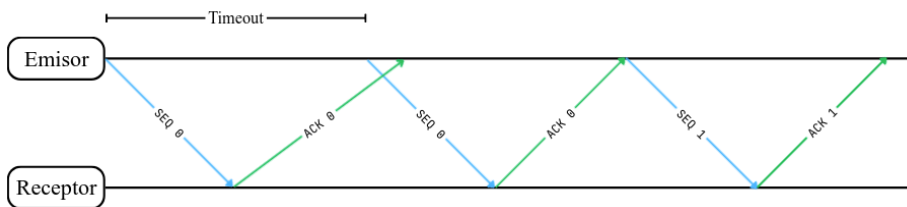


Figura 4: Retransmisión por acuse de recibo tardío.

El problema de Stop&Wait es que para archivos de gran tamaño el tiempo de transmisión será muy alto, debido al hecho que para poder enviar el siguiente segmento se debe esperar al ACK del paquete anterior. Por consiguiente, se debe encontrar una alternativa que optimice la cantidad de segmentos que se pueden enviar en un cierto periodo de tiempo que además asegure que todas las particiones del archivo lleguen sin problemas.

### 3.3.2. Selective Repeat

El Selective Repeat permite el control de errores a partir de la retransmisión de datos de los cuales no se recibió la confirmación de llegada, tal como se hace en Stop&Wait. La diferencia está en la característica de ventana de transmisión de Selective Repeat, donde se permite al emisor enviar múltiples paquetes de forma que si alguno de ellos falla en llegar, se tenga que retransmitir ese único fragmento de dato en vez de tener que esperar a que se envíen cada uno de los ACKs antes de enviar la siguiente oleada de segmentos.



The diagram illustrates the sliding window protocol with an Emissor and a Receptor. The Emissor sends packets (SEQ 0-11) and receives acknowledgments (ACK 0-11). The Receptor receives packets and stores them in a buffer. The buffer is divided into three sections: orange (0-3), purple (4-7), and grey (8-11). The orange section contains packets 0-3, the purple section contains packets 4-7, and the grey section contains packets 8-11. The buffer size is 12 packets.

ce para poder pasar a los siguientes paquetes por la Figura 5, dada una ventana de 4 paquetes, los blancos emiten mientras que los grises aún no se han recibido. Cuando se recibe el ACK del paquete de secuencia 0, se comienza a enviar el paquete de secuencia 4. Como en el caso de la Figura 5 no ocurren errores de ningún tipo, por cada paquete recibido se envía un nuevo paquete.

ocurra un error, ya sea que no llegó el paquete de recibo se perdió al enviarse, se retrasó o faltó llegar a alguna de las dos partes, dej

de la ventana a dicho paquete. Una vez se resuelve el error se retorna al envío normal, a menos que otro paquete se haya perdido en el periodo de retransmisión.

Como se muestra en la Figura 6, el paquete de secuencia 2 se pierde en el camino, por lo que el receptor de dicho paquete jamás envía su **ACK** correspondiente. Consecuentemente, dado que se acaba la ventana de *Timeout* de este paquete, se envía fuera de orden ahora si llegando a destino y enviándose el recibido de parte del receptor. Sin embargo, en el periodo de retransmisión se pierde el **ACK** correspondiente a la secuencia 5, por lo que el emisor vuelve a mandar el paquete a pesar de que el receptor ya lo tenga. En este caso, el receptor reenvía el acuse de recibo todas las veces que sean necesarias dependiendo de si vuelve a perderse dicho paquete de *acknowledgement* o no.

Dado a que puede darse el caso donde un paquete llegue fuera de orden, el receptor de los datos debe mantener un buffer en el que se vayan guardando los fragmentos ordenados para luego poder ir rearmando el archivo a partir de los datos recibidos.

### 3.4. Manejo de errores

La estrategia de manejo de errores se separa en dos categorías según en qué capa surge la falla. Por un lado, las pérdidas y corrupciones del medio se gestionan dentro del mecanismo de error-recovery descrito en la sección anterior; por el otro, las precondiciones lógicas y operativas (handshake mal formado, archivo inexistente, sin espacio en disco) se validan explícitamente y, ante un incumplimiento, se notifica al peer mediante un paquete con la flag **Error** activa que transporta el motivo en el payload.

Cada paquete recibido pasa primero por una función de *parsing* que verifica tres condiciones: que el datagrama tenga al menos los 9 bytes del header, que la longitud declarada en el campo **LENGTH** no exceda el payload realmente recibido, y que el checksum coincida con el valor recomputado sobre el header (con **CKSUM** en cero) más el payload. Si alguna falla, el paquete se descarta silenciosamente y el emisor lo retransmitirá tras el RTO, evitando que datos corruptos se confundan con datos válidos.

A nivel de aplicación, las precondiciones se centralizaron en una utilidad `validate(condicion, mensaje)` que ante un incumplimiento levanta una excepción `ValidationError`. Esta excepción se captura en los puntos de entrada del servidor y se traduce en el envío de un paquete con la flag **Error** hacia el cliente, conteniendo el mensaje de la condición violada. Se cubren,

entre otros casos, operaciones desconocidas, nombres de archivo vacíos o con bytes nulos, protocolos no soportados, y archivos cuya longitud declarada en el handshake supera el límite del servidor o no entraría en el almacenamiento disponible. La verificación de espacio se realiza antes de aceptar la sesión, consultando `os.statvfs` sobre el directorio de almacenamiento, para no comenzar una transferencia que terminaría abortada por `ENOSPC`.

Finalmente, los reintentos del protocolo están acotados por la constante `MAX_RETRIES`: si un paquete no es reconocido tras 20 intentos, el emisor abandona la transferencia con una excepción en lugar de bloquearse indefinidamente. La sesión se cierra y, si la falla ocurrió del lado del servidor, se reporta al cliente con un paquete `Error`. Todos los logs incluyen la dupla `[ip:puerto]` del peer involucrado, lo cual es necesario para diagnosticar problemas cuando hay múltiples sesiones concurrentes.

### 3.5. Concurrency

El servidor debe atender simultáneamente múltiples sesiones contra distintos clientes. Para resolver esto sin recurrir a un puerto distinto por cliente, se implementó un patrón *dispatcher* en el que un único socket UDP, atado al puerto público, recibe todos los datagramas y los re-rutea al thread responsable de la sesión correspondiente.

Concretamente, el módulo `server.dispatcher.Server` se queda en un loop leyendo del socket compartido y, por cada datagrama, identifica al peer por su tupla (IP, puerto) de origen. Si la dirección ya está asociada a una sesión activa, el paquete se encola en una cola dedicada de ese cliente (`queue.Queue`); si no lo está y el paquete es un `Handshake`, se valida su contenido y se instancia un `ClientHandler` (`threading.Thread`) que tomará control de la sesión. Cualquier paquete que no caiga dentro de ninguno de estos dos escenarios, se descarta y se loggea en nivel `DEBUG`, ya que típicamente corresponde a residuales de sesiones recién terminadas.

Cada `ClientHandler` no lee directamente del socket sino de su propia queue, mediante una función `recvfrom_fn` que se inyecta al protocolo Stop&Wait o Selective Repeat. El mismo código de protocolo, entonces, sirve indistintamente para el lado cliente (que lee del socket UDP directamente) y el lado servidor (que recibe los paquetes ya demultiplexados por el dispatcher), evitando duplicar la lógica de retransmisión.

La tabla que asocia direcciones a las queues se protege con un `threading.Lock`, ya que la modifican tanto el dispatcher al spawnear nuevas sesiones como un

thread auxiliar de cleanup que drena una queue `finished_q` en la que cada `ClientHandler` se anuncia al terminar. Esta liberación explícita evita una fuga de entradas en sesiones cortas y de alta concurrencia.

El uso de `threading` es debido a que las sesiones son I/O-bound, por lo que el modelo del interpreter de CPython no representa un cuello de botella, y el modelo de hilos se integra con la API bloqueante de `socket`.

## 4. Pruebas

Es importante conocer que la implementación del protocolo funciona de manera efectiva, eficiente y óptima. Para ello se utilizaron herramientas comunes en el ámbito de pruebas de protocolo de red para poder llevar al límite la función del protocolo FTP. Se hizo uso de la herramienta *mininet* para lograr simular una conexión entre dos nodos por fuera del *localhost* (de bloque de direcciones 127.0.0.0/8) así también pudiendo configurar un porcentaje de perdida de paquetes fija para tener un ambiente de pruebas estable que permita el correcto reconocimiento de las características del protocolo.

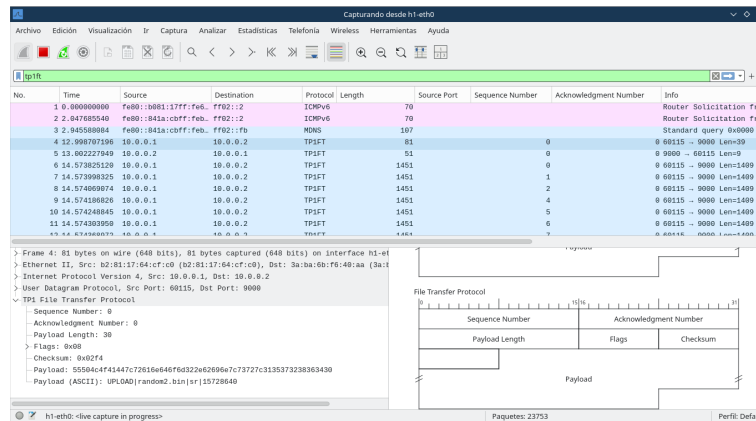


Figura 7: Vista de un paquete de Handshake a través del plugin de Wireshark.

Además, se utilizó la herramienta Wireshark para poder analizar con detenimiento el flujo de paquetes que se envían dentro de una transferencia de un archivo. Se creó un plug-in para la herramienta WireShark, permitiendo a esta poder identificar si un paquete es de este protocolo. En este plug-in se

renombró el protocolo a ‘TP1FT’ para evitar conflictos con otro protocolo del mismo nombre (FTP - RFC 959).

Al iniciar una captura de paquetes y aplicar el filtro correspondiente a TP1FT se puede observar tras una conexión entre Cliente y Servidor un grupo de paquetes formaron parte de la transferencia de un archivo, tal como se muestra en la Figura 7. A través de este plugin pueden verse las características que conforman al header-FTP y el payload traducido al formato ASCII, además de un pequeño diagrama de cómo está conformado todo el paquete de FTP.

Por ultimo el uso del argumento de salida verborrágica implementada permitió poder depurar el código desarrollado, además de poder comparar estadísticamente el funcionamiento de distintas configuraciones del protocolo. Al iniciar el servidor con la opción de salida verborragica se escriben en el archivo de *logs* del servidor una serie de lineas de información respecto al estado del propio servidor así como tambien de las conexiones que se realicen a él.

En particular se utilizará el mismo servidor para las pruebas de velocidad de transferencia de ambos mecanismos de error-recovery implementados. Por lo tanto, al iniciar el servidor se presenta el siguiente *log*:

```
mininet> h2 python3 src/start_server.py -v -H 10.0.0.2 -p
9000 -s /tmp/storage > /tmp/server.log
[2026-05-05 15:03:12] [INFO] SERVER - servidor escuchando en
10.0.0.2:9000
[2026-05-05 15:03:12] [INFO] SERVER - storage: /tmp/storage
```

Listing 1: Log de un servidor recién iniciado, esperando conexiones.

Habiendo presentado las herramientas usadas en las pruebas, se procede a explicar los procedimientos realizados y resultados obtenidos a partir de estas.

## 4.1. Stop&Wait

Una vez iniciado el servidor, el usuario puede decidir usar Stop&Wait para realizar la transferencia de su archivo. Utilizando el argumento del comando para admitir la salida verborrágica se presenta en pantalla un *log* sobre los paquetes enviados en la transferencia. Para una subida de un archivo de 10bytes como puede ser un archivo de texto que contenga ‘Hola Mundo’ en su interior, se vería por terminal lo que muestra el Listing 2:

```

mininet> h1 python3 src/upload.py -r sr -v -H 10.0.0.2 -p
9000 -s data/archivo.txt -n holamundo.txt
[2026-05-05 15:14:38] [INFO] UPLOAD - [10.0.0.2:9000] upload
'holamundo.txt' (0.00 MB) protocolo=stop_and_wait
[2026-05-05 15:14:38] [DEBUG] UPLOAD - [10.0.0.2:9000]
HANDSHAKE intento 1/5
[2026-05-05 15:14:40] [WARNING] UPLOAD - [10.0.0.2:9000]
HANDSHAKE timeout (intento 1/5)
[2026-05-05 15:14:40] [DEBUG] UPLOAD - [10.0.0.2:9000]
HANDSHAKE intento 2/5
[2026-05-05 15:14:40] [DEBUG] UPLOAD - [10.0.0.2:9000]
HANDSHAKE ok
[2026-05-05 15:14:40] [DEBUG] UPLOAD - [10.0.0.2:9000]
Archivo dividido en 1 chunks de hasta 1409 bytes
[2026-05-05 15:14:40] [DEBUG] UPLOAD - [10.0.0.2:9000]
Paquete enviado seq: 0 | chunk 1/1 | size: 10 bytes
[2026-05-05 15:14:40] [DEBUG] UPLOAD - [10.0.0.2:9000] ACK
recibido seq: 0
[2026-05-05 15:14:40] [DEBUG] UPLOAD - [10.0.0.2:9000] Nuevo
RTT adaptativo: 0.874s (RTT=0.001s)
[2026-05-05 15:14:40] [DEBUG] UPLOAD - [10.0.0.2:9000] FIN
enviado (intento 1/20)
[2026-05-05 15:14:40] [DEBUG] UPLOAD - [10.0.0.2:9000] FIN
ACK recibido
[2026-05-05 15:14:40] [INFO] UPLOAD - [10.0.0.2:9000]
transferencia completa en 0.00s

```

Listing 2: Log de Cliente Stop&Wait para una transferencia de un archivo pequeño.

Además puede observarse que el paquete del archivo dentro de WireShark. Donde puede observarse claramente que el payload se corresponde con el archivo como se muestra en la Figura 8.

Para verificar la velocidad de transferencia de este protocolo de control de errores, se creó un archivo binario de 15MB a partir del comando `urandom` que se enviará por método de Stop&Wait tanto por operatoria de carga como descarga del Cliente para poder compararlo posteriormente con el Selective Repeat.

Habiendo cargado el archivo a servidor se observa a través de la salida por pantalla del comando que, por Stop&Wait la carga tardó 159.32 segundos. Si bien casi 3 minutos de transferencia no parece mucho considerando que un 10% de los paquetes se pierden en vuelo, no logra superar el tiempo

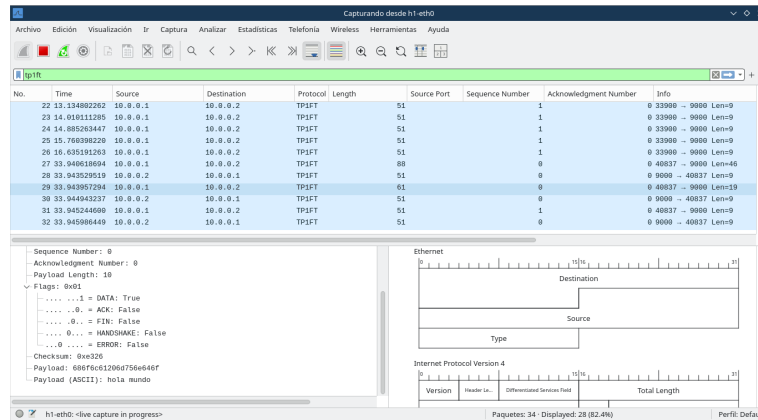


Figura 8: Vista del paquete “Hola Mundo” en Wireshark

límite impuesto para una transferencia de dicho tamaño y por lo tanto se debe probar el Selective Repeat para corroborar que esta alternativa no solo es más rápida en cuestiones de velocidad de transferencia sino que también logre alcanzar el límite superior establecido.

Una versión resumida de la salida del *upload* por Stop&Wait se muestra a continuación:

```
mininet> h1 python3 src/upload.py -v -H 10.0.0.2 -p 9000 -s
data/archivo_15mb.bin -n 15mb.bin
[2026-05-05 15:38:14] [INFO] UPLOAD - [10.0.0.2:9000] upload
'15mb.bin' (15.00 MB) protocolo=stop_and_wait
[2026-05-05 15:38:14] [DEBUG] UPLOAD - [10.0.0.2:9000]
HANDSHAKE intento 1/5
[2026-05-05 15:38:14] [DEBUG] UPLOAD - [10.0.0.2:9000]
HANDSHAKE ok
[2026-05-05 15:38:14] [DEBUG] UPLOAD - [10.0.0.2:9000]
Archivo dividido en 11235 chunks de hasta 1409 bytes
[2026-05-05 15:38:14] [DEBUG] UPLOAD - [10.0.0.2:9000]
Paquete enviado seq: 0 | chunk 1/11235 | size: 1400 bytes
[2026-05-05 15:38:14] [WARNING] UPLOAD - [10.0.0.2:9000] SW
sender: timeout esperando ACK seq=0, reintento 1/20
[2026-05-05 15:38:14] [DEBUG] UPLOAD - [10.0.0.2:9000]
Paquete enviado seq: 0 | chunk 1/11235 | size: 1400 bytes
[2026-05-05 15:38:14] [DEBUG] UPLOAD - [10.0.0.2:9000] ACK
recibido seq: 0
[2026-05-05 15:38:14] [DEBUG] UPLOAD - [10.0.0.2:9000] Nuevo
```

```

RT0 adaptativo: 0.874s (RTT=0.002s)
[2026-05-05 15:38:14] [DEBUG] UPLOAD - [10.0.0.2:9000]
  Paquete enviado seq: 1 | chunk 2/11235 | size: 1400 bytes
[2026-05-05 15:38:14] [DEBUG] UPLOAD - [10.0.0.2:9000] ACK
  recibido seq: 1
[2026-05-05 15:38:14] [DEBUG] UPLOAD - [10.0.0.2:9000] Nuevo
  RT0 adaptativo: 1.000s (RTT=0.001s)
[2026-05-05 15:38:14] [DEBUG] UPLOAD - [10.0.0.2:9000]
  Paquete enviado seq: 0 | chunk 3/11235 | size: 1400 bytes
...
[2026-05-05 15:40:46] [DEBUG] UPLOAD - [10.0.0.2:9000]
  Paquete enviado seq: 1 | chunk 11234/11235 | size: 1400
  bytes
[2026-05-05 15:40:46] [WARNING] UPLOAD - [10.0.0.2:9000] SW
  sender: timeout esperando ACK seq=1, reintento 1/20
[2026-05-05 15:40:46] [DEBUG] UPLOAD - [10.0.0.2:9000]
  Paquete enviado seq: 1 | chunk 11234/11235 | size: 1400
  bytes
[2026-05-05 15:40:46] [DEBUG] UPLOAD - [10.0.0.2:9000] ACK
  recibido seq: 1
[2026-05-05 15:40:46] [DEBUG] UPLOAD - [10.0.0.2:9000] Nuevo
  RT0 adaptativo: 0.050s (RTT=0.001s)
[2026-05-05 15:40:46] [DEBUG] UPLOAD - [10.0.0.2:9000]
  Paquete enviado seq: 0 | chunk 11235/11235 | size: 1040
  bytes
[2026-05-05 15:40:46] [DEBUG] UPLOAD - [10.0.0.2:9000] ACK
  recibido seq: 0
[2026-05-05 15:40:46] [DEBUG] UPLOAD - [10.0.0.2:9000] Nuevo
  RT0 adaptativo: 0.050s (RTT=0.001s)
[2026-05-05 15:40:46] [DEBUG] UPLOAD - [10.0.0.2:9000] FIN
  enviado (intento 1/20)
[2026-05-05 15:40:46] [DEBUG] UPLOAD - [10.0.0.2:9000] FIN
  enviado (intento 2/20)
[2026-05-05 15:40:46] [DEBUG] UPLOAD - [10.0.0.2:9000] FIN
  ACK recibido
[2026-05-05 15:40:46] [INFO] UPLOAD - [10.0.0.2:9000]
  transferencia completa en 159.32s

```

Listing 3: Resumen de log de upload Stop&Wait de 15MB

Además, para la descarga de un archivo a través de Stop&Wait se tardaron 149.71 segundos que tampoco logran superar la cota superior de tiempo de transmisión buscada para este trabajo. En el Listing 4 se muestra la salida de la descarga del archivo de 15MB del lado del cliente.



```

mininet> h1 python3 src/download.py -v -H 10.0.0.2 -p 9000 -d
/data/descargas -n 15mb.bin
[2026-05-05 15:44:49] [INFO] DOWNLOAD - [10.0.0.2:9000]
download '15mb.bin' > '/data/descargas/15mb.bin'
[2026-05-05 15:44:49] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
HANDSHAKE intento 1/5
[2026-05-05 15:44:49] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
HANDSHAKE ok
[2026-05-05 15:44:49] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
Datos recibidos seq: 0 | size: 1400 bytes
[2026-05-05 15:44:49] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
Datos recibidos seq: 1 | size: 1400 bytes
[2026-05-05 15:44:49] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
Datos recibidos seq: 0 | size: 1400 bytes
[2026-05-05 15:44:49] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
Datos recibidos seq: 1 | size: 1400 bytes
[2026-05-05 15:44:49] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
Datos recibidos seq: 0 | size: 1400 bytes
[2026-05-05 15:44:49] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
Datos recibidos seq: 1 | size: 1400 bytes
[2026-05-05 15:44:51] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
Paquete duplicado seq: 1 (esperado: 0) - re-ACK
[2026-05-05 15:44:51] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
Datos recibidos seq: 0 | size: 1400 bytes
[2026-05-05 15:44:52] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
Paquete duplicado seq: 0 (esperado: 1) - re-ACK
...
[2026-05-05 15:47:19] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
Datos recibidos seq: 0 | size: 1400 bytes
[2026-05-05 15:47:19] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
Datos recibidos seq: 1 | size: 1400 bytes
[2026-05-05 15:47:19] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
Datos recibidos seq: 0 | size: 1040 bytes
[2026-05-05 15:47:19] [DEBUG] DOWNLOAD - [10.0.0.2:9000] FIN
recibido - transferencia completa
[2026-05-05 15:47:19] [INFO] DOWNLOAD - [10.0.0.2:9000]
archivo recibido: /data/descargas/15mb.bin (15728640 bytes
) en 149.71s

```

Listing 4: Resumen de log de download Stop&Wait 15MB

## 4.2. Selective Repeat

Una vez habido obtenido resultados respecto al tiempo de transferencia de Stop&Wait, se contrastarán dichos datos con aquellos obtenidos de la ejecución del protocolo configurado para la repetición selectiva. Enviar el mismo archivo que contiene ‘Hola mundo’ a través de Selective Repeat resulta en el mismo log que con Stop&Wait del lado del Cliente.

La diferencia está en el momento de querer mandar un archivo de mayor tamaño, que requeriría dividirlo en chunks más pequeños. Enviando el mismo archivo de 15MB al Servidor se obtiene un resultado considerablemente diferente a que hacerlo con Stop&Wait.

```
mininet> h1 python3 src/upload.py -v -r selective_repeat -H
10.0.0.2 -p 9000 -s data/archivo_15mb.bin -n 15mbsr.bin
[2026-05-05 16:17:01] [INFO] UPLOAD - [10.0.0.2:9000] upload
'15mbsr.bin' (15.00 MB) protocolo=selective_repeat
[2026-05-05 16:17:01] [DEBUG] UPLOAD - [10.0.0.2:9000]
HANDSHAKE intento 1/5
[2026-05-05 16:17:01] [DEBUG] UPLOAD - [10.0.0.2:9000]
HANDSHAKE ok
[2026-05-05 16:17:01] [DEBUG] UPLOAD - [10.0.0.2:9000]
Archivo dividido en 11235 chunks de hasta 1409 bytes
[2026-05-05 16:17:03] [DEBUG] UPLOAD - [10.0.0.2:9000] SR
send seq=0 base=0
[2026-05-05 16:17:03] [DEBUG] UPLOAD - [10.0.0.2:9000] SR
send seq=1 base=0
[2026-05-05 16:17:03] [DEBUG] UPLOAD - [10.0.0.2:9000] SR
send seq=2 base=0
[2026-05-05 16:17:03] [DEBUG] UPLOAD - [10.0.0.2:9000] SR
send seq=3 base=0
[2026-05-05 16:17:03] [DEBUG] UPLOAD - [10.0.0.2:9000] SR
send seq=4 base=0
[2026-05-05 16:17:03] [DEBUG] UPLOAD - [10.0.0.2:9000] SR
send seq=5 base=0
[2026-05-05 16:17:03] [DEBUG] UPLOAD - [10.0.0.2:9000] SR
send seq=6 base=0
[2026-05-05 16:17:03] [DEBUG] UPLOAD - [10.0.0.2:9000] SR
send seq=7 base=0
[2026-05-05 16:17:03] [DEBUG] UPLOAD - [10.0.0.2:9000] SR
send seq=8 base=0
[2026-05-05 16:17:03] [DEBUG] UPLOAD - [10.0.0.2:9000] SR
send seq=9 base=0
[2026-05-05 16:17:03] [DEBUG] UPLOAD - [10.0.0.2:9000] SR
send seq=10 base=0
```

```

[2026-05-05 16:17:03] [DEBUG] UPLOAD - [10.0.0.2:9000] SR
    send seq=11 base=0
[2026-05-05 16:17:03] [DEBUG] UPLOAD - [10.0.0.2:9000] SR
    send seq=12 base=0
[2026-05-05 16:17:03] [DEBUG] UPLOAD - [10.0.0.2:9000] SR
    send seq=13 base=0
[2026-05-05 16:17:03] [DEBUG] UPLOAD - [10.0.0.2:9000] SR
    send seq=14 base=0
[2026-05-05 16:17:03] [DEBUG] UPLOAD - [10.0.0.2:9000] SR
    send seq=15 base=0
[2026-05-05 16:17:03] [DEBUG] UPLOAD - [10.0.0.2:9000] Nuevo
    RT0: 0.869s (RTT=0.008s)
[2026-05-05 16:17:03] [DEBUG] UPLOAD - [10.0.0.2:9000] SR ack
    seq=0 (base=0)
[2026-05-05 16:17:03] [DEBUG] UPLOAD - [10.0.0.2:9000] SR
    send seq=16 base=1
    ...
[2026-05-05 16:17:59] [WARNING] UPLOAD - [10.0.0.2:9000] SR
    sender: timeout seq=11232 -> reenvio (1/20)
[2026-05-05 16:17:59] [DEBUG] UPLOAD - [10.0.0.2:9000] Nuevo
    RT0: 0.050s (RTT=0.002s)
[2026-05-05 16:17:59] [DEBUG] UPLOAD - [10.0.0.2:9000] SR ack
    seq=11232 (base=11232)
[2026-05-05 16:17:59] [DEBUG] UPLOAD - [10.0.0.2:9000] FIN
    enviado (intento 1/20)
[2026-05-05 16:17:59] [DEBUG] UPLOAD - [10.0.0.2:9000] FIN
    ACK recibido
[2026-05-05 16:17:59] [INFO] UPLOAD - [10.0.0.2:9000]
    transferencia completa en 57.73s

```

Listing 5: Resumen de log de carga Selective Repeat 15MB

Haciendo uso de Selective Repeat con una ventana de 16 paquetes consecutivos se logra superar (y para nada marginalmente) la cota superior de 120 segundos de tiempo de transferencia, con un tiempo final de 57.73 segundos. Lo mismo ocurre en la descarga donde se logra descargar el archivo de 15MB en tan solo 54.24 segundos.

```

[2026-05-05 16:48:12] [INFO] DOWNLOAD - [10.0.0.2:9000]
    download '15mbsr.bin' > '/data/descargas/15mbsr.bin'
[2026-05-05 16:48:12] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
    HANDSHAKE intento 1/5
[2026-05-05 16:48:12] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
    HANDSHAKE ok
[2026-05-05 16:48:12] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
    Iniciando recepcion en /data/descargas/15mbsr.bin

```

```

[2026-05-05 16:48:14] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
    Paquete 0 en ventana - ACK + buffer
[2026-05-05 16:48:14] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
    Entregando paquete 0
[2026-05-05 16:48:14] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
    Paquete 1 en ventana - ACK + buffer
[2026-05-05 16:48:14] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
    Entregando paquete 1
    ...
[2026-05-05 16:48:15] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
    Entregando paquete 26
[2026-05-05 16:48:15] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
    Paquete 18 antiguo (duplicado) - re-ACK
[2026-05-05 16:48:15] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
    Paquete 19 antiguo (duplicado) - re-ACK
    ...
[2026-05-05 16:49:07] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
    Paquete 11230 antiguo (duplicado) - re-ACK
[2026-05-05 16:49:07] [DEBUG] DOWNLOAD - [10.0.0.2:9000] FIN
    recibido. Enviando ACK para FIN.
[2026-05-05 16:49:07] [DEBUG] DOWNLOAD - [10.0.0.2:9000]
    Recepcion finalizada.
[2026-05-05 16:49:07] [INFO] DOWNLOAD - [10.0.0.2:9000]
    archivo recibido: /data/descargas/15mbsr.bin (15728640
    bytes) en 54.24s

```

Listing 6: Resumen de log de descarga Selective Repeat 15MB

Puede observarse claramente el funcionamiento del Selective Repeat en acción en ambos Listings. A partir del Listing 5, se puede observar como en principio se envían una cantidad específica de paquetes antes de esperar un ACK del receptor. Además, en el Listing 6, puede observarse como cada paquete recibido va moviendo la ventana de recibos. Una vez se recibe un paquete duplicado (cualquiera sea la razón) se envía otra vez el acuse de recibo para notificar al emisor. Por último, el emisor envía un paquete tipo FIN para avisar al receptor que no llegarán más paquetes a lo que el receptor envía el correspondiente ACK, cerrando la conexión.

### 4.3. Resultados

Como pudo observarse, a partir de los datos obtenidos, el Selective Repeat resulta ser casi tres veces más rápido que el Stop&Wait, configurado con una ventana de 16, no solo superando en velocidad a la otra configuración, sino

que además logrando superar el límite de tiempo estimado para la transmisión de un archivo de 15MB de tamaño. Esto demuestra que, si bien para ciertas circunstancias el método de control de errores de transmisión no afecta a la velocidad con la que se transmite el paquete, existen otros casos más cercanos a la realidad donde la elección correcta de que protocolo a utilizar resulta ser menos marginal y por lo tanto puede dar resultados significativamente diferentes.

## 5. Preguntas a responder

En esta sección se definirán algunos conceptos que a pesar de no tener relación con el aspecto técnico del protocolo implementado, vale la pena describirlos para profundizar un poco en ellos así mejorar el entendimiento teórico de los temas tratados en este informe.

### 5.1. Modelo Cliente-Servidor

En la descripción de la implementación se describió lo siguiente: ‘El protocolo FTP se basa en el Modelo Cliente-Servidor’, pero *¿Que es el Modelo Cliente-Servidor?*

Este modelo es una arquitectura de diseño de software donde las tareas se dividen entre los proveedores de servicios (Servidores) y quienes demandan dichos servicios (Clientes). Los Servidores esperan pasivamente a que un cliente comience la conexión con ellos y su función es la de procesamiento de pedidos y almacenamiento de datos. Mientras tanto, los Clientes se conectan a los Servidores y realizan peticiones a estos sirviendo de interfaz de comunicación para el usuario.

A diferencia del modelo peer-to-peer (P2P), este modelo tiene los roles bien marcados respecto a qué dispositivo o proceso sirve como Servidor y cual como Cliente. Además, se presenta una diferencia considerable respecto a las características que representa a cada uno. Los Servidores suelen tenerse en cantidades considerablemente menores que los clientes que se conectan a ellos y suelen tener potencia de procesamiento mucho mayor a los dispositivos que realizan peticiones a ellos. En contraposición, en el modelo P2P puede ocurrir que ambos dispositivos conectados tengan características marginalmente diferentes dado a que los roles no se encuentran tan marcados.

## 5.2. Función de un Protocolo de Aplicación

Agregando a lo mencionado anteriormente, se explicó que FTP es un ‘protocolo de capa de aplicación’, y si bien la descripción de la implementación del protocolo responde indirectamente sobre qué significa esta categorización del protocolo, vale la pena responder a la pregunta *¿Cuál es la función de un Protocolo de Aplicación?*

Si el modelo Cliente-Servidor describe la relación de un proveedor de servicios y uno o más demandantes de dicho servicios, un protocolo de capa de aplicación sirve de intermediario a los requerimientos de ambas partes del modelo.

En particular, el protocolo de capa de aplicación define cuatro aspectos importantes en la conexión. En principio indica qué tipo de mensaje se envía en la comunicación entre ambos participantes de la conexión, esto significa que dado un grupo de tipos de mensajes el protocolo de capa de aplicación determina cual de todos esos mensajes es el que se envió. En segundo lugar, define la sintaxis del mensaje a enviarse, es decir de qué manera se conforma el mensaje. Por ejemplo, podría ocurrir que en un paquete particular, los primeros 100 bits representan una operación a hacerse con los siguientes 100 bits. En este caso, el protocolo de aplicación tendría la función para informar a quien recibe el paquete qué representan estos 200 bits.

Además estos protocolos definen qué significa cada valor que puede tomar los campos de la estructura del mensaje a enviarse, también conocido como semántica. Esta característica funciona de “diccionario” para ambas partes de la conexión, exponiendo la función de cada campo del paquete a quien lo recibe. Volviendo al ejemplo de los 100 bits de operatoria, podría ocurrir que el valor 50 en binario corresponde a la operación booleana **NOT** con los 100 bits que le siguen. El protocolo de aplicación sabría reconocer que dicho valor en ese campo corresponde a la operatoria mencionada y permitiría al receptor del paquete saber qué hacer gracias a tener una semántica configurada.

Por último, el protocolo de aplicación determina las reglas de tiempo de envío de paquetes entre emisor y receptor. Esto quiere decir que configura las reglas de como y cuando uno de los participantes puede y/o debe enviar un paquete. Por ejemplo, para la implementación de Selective Repeat se determinó que dado un valor  $w$  de ventana, se pueden enviar hasta  $w$  paquetes sin necesidad de tener que recibir un **ACK** de parte del receptor, mientras que en el Stop&Wait por cada paquete que envía el emisor, el receptor debe de enviar correctamente el acuse de recibo para que el emisor pueda volver a

mandarle otro segmento de archivo.

### 5.3. Otras características de FTP

En el presente trabajo se ha diseñado un protocolo de aplicación simplificado para la transferencia de archivos, operando sobre una capa de transporte personalizada que utiliza *Selective Repeat* sobre *UDP*. A diferencia de protocolos complejos como *HTTP* o el *FTP* estándar (RFC 959), este protocolo se centra en la eficiencia de la transferencia unidireccional y la gestión mínima de estados.

Debido al hecho que, en caso de usar *Selective Repeat*, la ventana de envío se mantiene fija, este protocolo no implementa un control de congestión ya que para los fines de una transferencia punto a punto en una red aislada, el control de flujo es suficiente para evitar el desborde de los buffers finales.

#### 5.3.1. Sintaxis de los mensajes

Para garantizar que ambos extremos interpreten correctamente el flujo de bytes, se definió una estructura de mensajes de control previa a la ráfaga de datos. Antes de realizar la entrega de datos del archivo se envía dentro del paquete de **Handshake** un grupo de características del archivo a enviarse:

1. **OPERATION**: Puede ser **UPLOAD** o **DOWNLOAD**, sirve para dejar sentado los roles del Cliente y Servidor.
2. **filename**: Describe el nombre del archivo a guardarse ya sea dentro del Servidor o del Cliente.
3. **protocol**: Indica si se desea utilizar **Stop&Wait** o **Selective Repeat** en la ráfaga de segmentos de archivos a enviarse.
4. **file\_size**: Representa el tamaño del archivo a enviarse, sirve para determinar si el archivo cabe en el recipiente. En caso de no entrar se envía un paquete de tipo **Error** dentro del periodo de *Handshake*

Adicionalmente, el protocolo admite respuestas de error de aplicación, como el código de “*Archivo no encontrado*”, que permite al servidor abortar la conexión de forma limpia antes de inicializar la ventana de transporte si el recurso solicitado no existe.

### 5.3.2. Estrategia de fragmentación

La capa de aplicación es la encargada de realizar el *framing* del archivo binario. El archivo se segmenta en trozos (chunks) de tamaño fijo de 1400 bytes. Esta decisión de diseño busca optimizar el encapsulamiento. Al sumar los *headers* de transporte (SR), red (IP) y enlace (Ethernet), el paquete resultante se mantiene por debajo de la MTU estándar (1500 bytes), evitando la fragmentación en capas inferiores que degradaría el rendimiento del algoritmo *Selective Repeat*.

### 5.3.3. Maquina de estados y ciclo de vida

Finalmente, el flujo lógico del protocolo se divide en tres etapas claramente diferenciadas:

1. **Handshake:** Se realiza un *handshake* simplificado donde se negocian los parámetros de la transferencia y se verifica la disponibilidad del archivo.
2. **Transferencia:** La aplicación entrega de forma secuencial los segmentos al motor de *Selective Repeat* o *Stop&Wait*. En esta fase, la aplicación se mantiene a la espera de que el transporte confirme la entrega fehaciente de cada bloque.
3. **Fase de Sincronización de Cierre:** Una vez que el emisor ha transmitido el último segmento, envía un paquete especial de control (*FIN*). La conexión se considera finalizada únicamente cuando el receptor confirma el *FIN* mediante un *ACK* de aplicación, asegurando que todos los datos han sido volcados desde el *buffer* de memoria hacia el almacenamiento persistente.

## 5.4. Protocolo UDP vs. TCP

*TCP* (RFC 793) es un protocolo de capa de transporte orientado a la conexión: antes de transmitir datos las dos partes establecen una sesión vía un *three-way handshake*, durante el cual se sincronizan los números de secuencia iniciales. Los segmentos se ordenan en el receptor antes de entregarse a la aplicación, las pérdidas se detectan con *ACKs* acumulativos y se retransmiten automáticamente, y dos mecanismos adicionales: control de flujo (basado



en una ventana anunciada por el receptor) y control de congestión (*slow start, congestion avoidance, fast retransmit/recovery*), que regulan el caudal de envío para no saturar al receptor ni a la red. El header de TCP ocupa al menos 20 bytes.

*UDP* (RFC 768) tiene 8 bytes de header, sin establecimiento de conexión, sin garantías de orden, sin retransmisión, sin control de flujo y sin control de congestión. Cada datagrama se trata de manera independiente y la aplicación es la responsable de manejar pérdidas, reordenamientos o congestión.

### 5.5. Stop&Wait vs. Selective Repeat

Tal como se mostró en secciones anteriores, el Selective Repeat resulta ser un protocolo de recuperación de errores mucho más veloz en sentido de tiempo de transferencia que el Stop&Wait, pero vale la pena preguntarse si *¿Es siempre más veloz usar Selective Repeat que Stop&Wait?*

Basando el análisis en un caso borde, se quiere enviar un archivo pequeño (menor al tamaño de partición en segmentos determinada, en particular menor a 1400 bytes). Si se envía este archivo a través del protocolo implementado utilizando cualquiera de los dos mecanismos de error-recovery, se realizará la misma operatoria. El emisor del archivo mandará el paquete que contenga en su payload la totalidad del archivo, esperará a recibir el acuse de recibo y finalmente cerrará la conexión. Por lo tanto considerando las características de ambas implementaciones, se puede asegurar que el uso de Stop&Wait en circunstancias como la descrita es mucho más beneficioso dado su simpleza de implementación.

Otra situación donde el Selective Repeat resulta en un Stop&Wait más complejo es en el caso donde se configura una ventana de 1 en la repetición selectiva. En consecuencia a realizar esta configuración, el mecanismo de Selective Repeat funcionará en todo momento como Stop&Wait, eliminando cualquier beneficio que pueda presentarse en la implementación de dicho protocolo de control de errores de transferencia.

## 6. Dificultades encontradas

El desarrollo del proyecto presentó una serie de desafíos que trascendieron lo estrictamente técnico, situándose principalmente en la gestión de recursos

humanos y la administración del tiempo. A continuación, se detallan las principales dificultades enfrentadas.

La mayor dificultad externa fue la coincidencia del cronograma de desarrollo con un periodo de alta carga evaluativa en otras asignaturas de la carrera. Esta superposición generó cuellos de botella en las fases intermedias del proyecto, limitando la disponibilidad de tiempo para sesiones de programación conjunta y depuración de errores en profundidad. La necesidad de priorizar compromisos académicos paralelos obligó a compactar las etapas de pruebas y documentación en periodos de tiempo más reducidos de lo inicialmente previsto.

A nivel técnico, la transición del desarrollo en *localhost* hacia el entorno de red virtualizado en *Mininet* introdujo variables de red reales (latencia y pérdida de paquetes) que dificultaron la identificación de errores de concurrencia. Depurar el comportamiento de la ventana de recepción frente a duplicados y paquetes fuera de orden demandó un análisis exhaustivo de los registros (*logs*), resultando en una curva de aprendizaje pronunciada sobre el uso de herramientas de diagnóstico y la gestión de permisos en entornos Linux.

Para superar estos obstáculos, el equipo debió adoptar un enfoque de comunicación más asertiva, estableciendo hitos de entrega internos más frecuentes y utilizando herramientas de control de versiones para mitigar la falta de presencialidad. La resolución de los conflictos de permisos y la optimización de la ventana de transmisión permitieron alcanzar los objetivos de rendimiento propuestos, transformando las dificultades de gestión en una experiencia de aprendizaje sobre la planificación de proyectos de ingeniería en entornos de alta demanda.

## 6.1. Posibles mejoras

Creemos que la implementación deja varias optimizaciones como trabajo a futuro.

- **Regla de Karn:** actualmente la muestra de RTT se toma incluso cuando el paquete fue retransmitido, lo cual es ambiguo (no se sabe si el ACK responde a la transmisión original o a una retransmisión) y puede sesgar el RTO. La regla de Karn descarta las muestras provenientes de paquetes retransmitidos y duplica el RTO ante cada timeout (*exponential backoff*).

- **ACKs acumulativos / SACK:** nuestro Selective Repeat envía un ACK por cada paquete recibido. TCP usa ACKs acumulativos y, mediante la opción *Selective Acknowledgement* (RFC 2018), puede reportar bloques no contiguos en un único ACK, reduciendo el tráfico de control en el camino inverso.
- **Fast retransmit:** TCP retransmite un segmento al recibir tres ACKs duplicados sin esperar el timeout, acelerando la detección de pérdidas. Se podría adaptar a nuestro esquema de re-ACK del receptor.
- **Control de congestión:** la ventana actual es estática. Algoritmos como *TCP Reno* o *CUBIC* ajustan la ventana dinámicamente, expandiéndose mientras la red lo permita y contrayéndose ante pérdidas, evitando saturar enlaces compartidos.
- **Cifrado:** la transferencia es en texto plano. DTLS (RFC 6347) provee TLS sobre datagramas y podría aplicarse sin necesidad de refactorizar tanto la implementación actual.

## 7. Conclusión

El desarrollo y la implementación del protocolo de transferencia de archivos sobre UDP permitieron profundizar en la mecánica interna de la capa de transporte y su interacción con la capa de aplicación. A lo largo de este trabajo, se demostró que la confiabilidad en redes no orientadas a la conexión no es una característica intrínseca del canal, sino una responsabilidad del diseño lógico de los protocolos.

Los resultados obtenidos mediante las pruebas en *Mininet* confirmaron la hipótesis inicial: el mecanismo de *Selective Repeat* presenta una eficiencia significativamente superior al de *Stop&Wait* en escenarios de red con pérdida de paquetes. Mientras que *Stop&Wait* se ve limitado por el tiempo de ida y vuelta (RTT) en cada segmento enviado, la capacidad de *Selective Repeat* para mantener una ventana de transmisión activa permitió maximizar el uso del ancho de banda disponible, logrando transferir el archivo de 15 MB dentro del límite temporal establecido de dos minutos, incluso bajo una tasa de pérdida del 10 %.

En conclusión, este proyecto subraya la importancia de la selección de protocolos según los requerimientos de la aplicación y las condiciones del en-

torno. Para un ingeniero informático, entender estas sutilezas es fundamental; no solo para utilizar las herramientas existentes, sino para diseñar nuevas soluciones capaces de escalar y resistir las imperfecciones de las redes reales. La experiencia de implementar estas capas desde cero brinda una perspectiva invaluable sobre el modelo TCP/IP, transformando conceptos teóricos abstractos en herramientas prácticas de ingeniería.

## 8. Referencias

1. Kurose, J. F., & Ross, K. W. (2020). *Computer Networking: A Top-Down Approach* (8ª ed.). Pearson.
2. Stevens, W. R., & Fall, K. R. (2011). *TCP/IP Illustrated, Volume 1: The Protocols* (2ª ed.). Addison-Wesley.
3. Postel, J. (1980). *User Datagram Protocol*. RFC 768. <https://www.rfc-editor.org/rfc/rfc768>
4. Postel, J. (1981). *Transmission Control Protocol*. RFC 793. <https://www.rfc-editor.org/rfc/rfc793>
5. Postel, J., & Reynolds, J. (1985). *File Transfer Protocol (FTP)*. RFC 959. <https://www.rfc-editor.org/rfc/rfc959>
6. Braden, R., Borman, D., & Partridge, C. (1988). *Computing the Internet Checksum*. RFC 1071. <https://www.rfc-editor.org/rfc/rfc1071>
7. Mathis, M., Mahdavi, J., Floyd, S., & Romanow, A. (1996). *TCP Selective Acknowledgment Options*. RFC 2018. <https://www.rfc-editor.org/rfc/rfc2018>
8. Paxson, V., Allman, M., Chu, J., & Sargent, M. (2011). *Computing TCP's Retransmission Timer*. RFC 6298. <https://www.rfc-editor.org/rfc/rfc6298>
9. Rescorla, E., & Modadugu, N. (2012). *Datagram Transport Layer Security Version 1.2*. RFC 6347. <https://www.rfc-editor.org/rfc/rfc6347>
10. Python Software Foundation. *The Python Standard Library*. <https://docs.python.org/3/library/>
  - `socket` for low-level networking interface: <https://docs.python.org/3/library/socket.html>
  - `struct` to interpret bytes as packed binary data: <https://docs.python.org/3/library/struct.html>
  - `threading` for thread-based parallelism: <https://docs.python.org/3/library/threading.html>

- queue class: <https://docs.python.org/3/library/queue.html>
  - logging: <https://docs.python.org/3/library/logging.html>
11. Mininet Project. *Mininet: An Instant Virtual Network on your Laptop (or other PC)*. <http://mininet.org>